DEVELOPMENT ROADMAP

DEVELOPMENT_ROADMAP.md

Milestones sized at a few hours to one day, mapped loosely to a 15-week, 4-release college schedule. No implementation here — planning only. Actual per-session breakdown into 30–90 minute chunks happens live with the AI mentor per `AI_MENTOR_INSTRUCTIONS.md`.

Release 1 (Weeks 1–4): Foundations — Data & Dashboard Skeleton

Milestone 1.1 — Project Setup

- **Goal:** Repo, folder structure, virtual environment, `requirements.txt`, Postgres instance provisioned (hosted, e.g. Supabase free tier).
- **Concepts learned:** Environment management, secrets/.env handling, git basics for a team.
- **Files/modules involved:** Root config files, `.env.example`, `db/connection.py`.
- **Expected output:** Empty Streamlit app that runs and successfully connects to the hosted Postgres instance.
- **Common mistakes:** Committing real secrets; not git-ignoring `.env` / `secrets.toml`.
- **Prerequisites:** None.
- **Completion criteria:** `streamlit run dashboard/app.py` runs; a test query against Postgres succeeds.

Milestone 1.2 — Database Schema

- **Goal:** Implement `schema.sql` for `transactions`, `merchant_cache`, `insights_history` per `DATABASE_DESIGN.md`.
- **Concepts learned:** DDL, primary/foreign keys, indexes.
- **Files/modules involved:** `db/schema.sql`.
- **Expected output:** Tables exist in the hosted DB, verified via a client (e.g., `psql` or a GUI).
- **Common mistakes:** Forgetting indexes; wrong column types for money (should be NUMERIC, not FLOAT).
- **Prerequisites:** 1.1.
- **Completion criteria:** All three tables created and queryable.

Milestone 1.3 — CSV Upload + Validation

- **Goal:** Build the Upload page and `utils/validation.py`.
- **Concepts learned:** Pandas dtype handling, Streamlit file uploads, structured error messaging.
- **Files/modules involved:** `dashboard/pages/1_Upload.py`, `utils/validation.py`.
- **Expected output:** Valid CSVs preview correctly; invalid CSVs show clear errors.
- **Common mistakes:** Only checking column names, not types; not handling empty files.
- **Prerequisites:** 1.1.
- **Completion criteria:** Demonstrated with both a valid and an intentionally broken sample CSV.

Milestone 1.4 — Data Cleaning

- **Goal:** Implement `pipeline/cleaning.py`.
- **Concepts learned:** Regex normalization, deduplication logic.
- **Files/modules involved:** `pipeline/cleaning.py`, `tests/test_cleaning.py`.
- **Expected output:** Cleaned DataFrame with a `cleaned_description` column.
- **Common mistakes:** Over-aggressive regex stripping useful merchant info.
- **Prerequisites:** 1.3.
- **Completion criteria:** Unit tests pass on sample messy strings.

Release 2 (Weeks 5–8): Categorization Pipeline

Milestone 2.1 — Dictionary + Fuzzy Matching (Tiers 2–3)

- **Goal:** Implement dictionary lookup and rapidfuzz matching.
- **Concepts learned:** String matching algorithms, Levenshtein distance, threshold tuning.

- **Files/modules involved:** `pipeline/categorization/tier2_dictionary.py`, `tier3_fuzzy.py`.
- **Expected output:** Correctly categorizes known and near-known merchants.
- **Common mistakes:** Threshold too loose (false positives) or too strict (everything falls through).
- **Prerequisites:** 1.4.
- **Completion criteria:** Tests cover exact, near-miss, and non-match cases.

Milestone 2.2 — Merchant Cache (Tier 0)

- **Goal:** Implement `merchant_cache` read/write.
- **Concepts learned:** Caching patterns, upsert logic.
- **Files/modules involved:** `db/queries.py`, `pipeline/categorization/tier0_cache.py`.
- **Prerequisites:** 1.2, 2.1.
- **Completion criteria:** Repeat merchant lookups skip Tiers 1–5 and hit cache.

Milestone 2.3 — TF-IDF + Logistic Regression Classifier (Tier 1)

- **Goal:** Train and integrate a lightweight classifier as the first-pass tier.
- **Concepts learned:** TF-IDF vectorization, logistic regression, train/test split, evaluation metrics (precision/recall/F1), model serialization.
- **Files/modules involved:** `notebooks/model_training.ipynb`, `models/tfidf_logreg.pkl`, `pipeline/categorization/tier1_classifier.py`.
- **Prerequisites:** 2.1 (need labeled data, which the dictionary helps bootstrap).
- **Completion criteria:** Model trained, evaluated, exported, and integrated into the pipeline with a confidence threshold.

Milestone 2.4 — Embedding Similarity (Tier 4)

- **Goal:** Integrate sentence-transformers for semantic fallback matching.
- **Concepts learned:** Embeddings, cosine similarity, vector representations of text.
- **Files/modules involved:** `pipeline/categorization/tier4_embedding.py`.
- **Prerequisites:** 2.1.
- **Completion criteria:** Semantically similar but textually different merchants correctly categorized.

Milestone 2.5 — LLM Fallback (Tier 5) + Orchestration

- **Goal:** Wire up the LLM API as final fallback; build `pipeline.py` to orchestrate all tiers in order.
- **Concepts learned:** LLM API integration, prompt engineering, structured output parsing, error handling for external calls.
- **Files/modules involved:** `pipeline/categorization/tier5_llm.py`, `pipeline/categorization/pipeline.py`.
- **Prerequisites:** 2.1–2.4.
- **Completion criteria:** Full tier chain runs end-to-end on a real sample CSV; every transaction gets a category.

Release 3 (Weeks 9–11): Emissions, Dashboard, Insights

Milestone 3.1 — Emission Calculation

- **Goal:** Implement `pipeline/emissions.py` with DEFRA/EPA factors + PPP adjustment.
- **Concepts learned:** Spend-based emissions methodology, unit conversion, PPP adjustment logic.
- **Prerequisites:** Release 2 complete.
- **Completion criteria:** Correct kgCO2e values produced and spot-checked against manual calculation.

Milestone 3.2 — Dashboard: Metrics + Trend + Breakdown

- **Goal:** Build `2_Dashboard.py` and chart/metric components.
- **Concepts learned:** Streamlit layout, Plotly charting, `st.cache_data`.
- **Prerequisites:** 3.1.
- **Completion criteria:** Dashboard renders real data with correct totals and trends.

Milestone 3.3 — AI Insight Generation

- **Goal:** Implement `pipeline/insights.py` and the Insights page.

- **Concepts learned:** Prompt construction from aggregated data, caching LLM output.
- **Prerequisites:** 3.2.
- **Completion criteria:** Insight text generated, cached, and displayed with graceful failure handling.

Release 4 (Weeks 12–15): Polish, Testing, Deployment

Milestone 4.1 — Testing Suite

- **Goal:** Fill out `tests/` per `TESTING.md`.
- **Prerequisites:** All prior milestones.
- **Completion criteria:** `pytest` passes across cleaning, categorization, emissions, and integration tests.

Milestone 4.2 — Design System Application

- **Goal:** Apply `FRONTEND_DESIGN_SYSTEM.md` consistently (theming, empty/loading states).
- **Prerequisites:** 3.2, 3.3.
- **Completion criteria:** All screens match the design system's wireframes/palette.

Milestone 4.3 — Deployment

- **Goal:** Deploy to Streamlit Community Cloud with hosted Postgres and secrets configured.
- **Prerequisites:** 4.1.
- **Completion criteria:** Publicly accessible, working demo link.

Milestone 4.4 — Documentation & Presentation Polish

- **Goal:** Finalize `README.md`, prepare demo walkthrough, review `INTERVIEW_PREP.md` material.
- **Prerequisites:** 4.3.
- **Completion criteria:** Repo is presentable; team can explain every architectural decision.